YOLOv8-Based License Plate Recognition for Bangladeshi Vehicles

The paper, our re-run on dataset v1.1, why the detector is fine but EasyOCR is not, what the numbers mean for our system, and what to show and say to the faculty

Prepared for MD. Rokib Hasan Oli (paper reader) · Supervisor: Dr. Md Mansur ul Mulk · 24 October 2026
Paper: I. Ahamed and G. M. Ismail, Authorea preprint (not peer-reviewed), posted 21 Jan 2025, doi 10.22541/au.173748276.69471512/v1; 22 pages, text on p.1-17. Page numbers are PDF pages.
Code and data: none published. Our re-implementation: Ultralytics YOLOv8 8.4.170 + EasyOCR 1.7.2 (`paper06_kaggle.ipynb`).
Our run: Tesla T4 x2 on Kaggle, 24 Oct 2026, dataset **v1.1** (confirmed in `environment.json`), 2.8 h; full output in `After Train/results.zip`. Extra checks run locally the same day (sections 10-13).

The whole report on one page

The paper in three lines. A COCO-pretrained YOLOv8 is fine-tuned to find Bangladeshi number plates in 2,600 own photos, and the stock EasyOCR Bangla model reads the cropped plate. The authors report precision 93.5 %, recall 92.3 %, mAP 94.0 % and 78 FPS for the detector, and 92.7 % "character recognition accuracy" for EasyOCR (p.9-12). No metric is defined, the photos were augmented before the split, and no code, data or settings are given.

Is the v1.1 training OK? Yes. All 7 jobs ended with `code=0`, dataset version 1.1 is confirmed, both detectors ran all 120 epochs with no NaN (best epochs 90 and 79), and the run took 2.8 h of the 12 h limit. A local re-check with the run's `best.pt` files gives the Kaggle test numbers to the last digit (mAP50 97.39 and 97.62), and a local EasyOCR re-run gives the same character accuracy (60.45 %). The one number that looks wrong, mAP50-95 = 45.0 (paper about 59), is explained by one test source with tighter label boxes, not by the training (section 10).

Five findings. (1) **The detector matches the paper and our targets:** YOLOv8n mAP50 97.4 %, YOLOv8s 97.6 % (paper 94.0 %, proposal target 95 %); 96.4 / 97.1 % of the 1,011 test plates found. (2) **mAP50-95 is a labelling effect:** the ridoykj source (274 test photos) is boxed tightly; it scores 22.1 there and 64.3 without it. (3) **EasyOCR is the weak link:** 60.5 % character accuracy (paper 92.7 %) and only **13.5 %** of 651 plates read exactly; 39 % of its readings contain English digits, and the paper-style preprocessing makes it worse (4.9 %). (4) **Swap only the reader and the system works:** on the same 204 whole photos with the same YOLOv8 crops, a character detector (paper #2's char-YOLO12n) reads 84.8 % (v8n) and 88.2 % (v8s) exactly instead of 11.8 / 10.3 % with EasyOCR. (5) **Speed is no problem:** YOLOv8n runs at 114 FPS in PyTorch FP32 and 183 FPS with TensorRT FP16 on a T4 (paper 78 FPS); EasyOCR costs 90.9 ms per plate, 17x the detector.

What to tell the faculty in one sentence. "We rebuilt this paper on our duplicate-free benchmark: its YOLOv8 plate detector is as good as claimed (97 % mAP50), but stock EasyOCR reads only 14 % of real Bangladeshi plates correctly, so in our system YOLOv8 stays as the detector and EasyOCR stays only as the baseline that our character-level reader must beat, which it does with 88 % on the same plates."

How this report is organised

Part	Sections	Read it when
A. Understand the paper	1-6: problem, terms, data, method, results, weaknesses	before you explain the paper
B. Our v1.1 experiment	7-15: what we ran, is it OK, results vs paper, detector, EasyOCR, fixes, whole system, speed, findings	before you quote any number
C. Use it	16-22: research use, benefits, faculty slides, script, Q&A, next steps, GitHub/ Drive	before the faculty meeting

1. What problem does the paper solve?

Bangladesh has more than 4.8 million registered vehicles and low-resolution CCTV, and its plates have two lines with Bangla words, a series letter and six Bangla digits (p.1). The paper builds an automatic plate reader for these plates: **find** the plate in a street photo, then **read** its Bangla text. It claims to work under low resolution, rain, dust, night and partly hidden plates (p.3, p.12-13).

The method is the simplest full pipeline possible with off-the-shelf tools: a COCO-pretrained **YOLOv8** fine-tuned as a one-class plate detector, then the stock **EasyOCR** Bangla model on the plate crop (p.5-8). Nothing new is trained for reading. That simplicity is exactly why it matters for us: it is the baseline our proposal promised ("EasyOCR after our detector", proposal section on baselines), and every better reader must beat it.

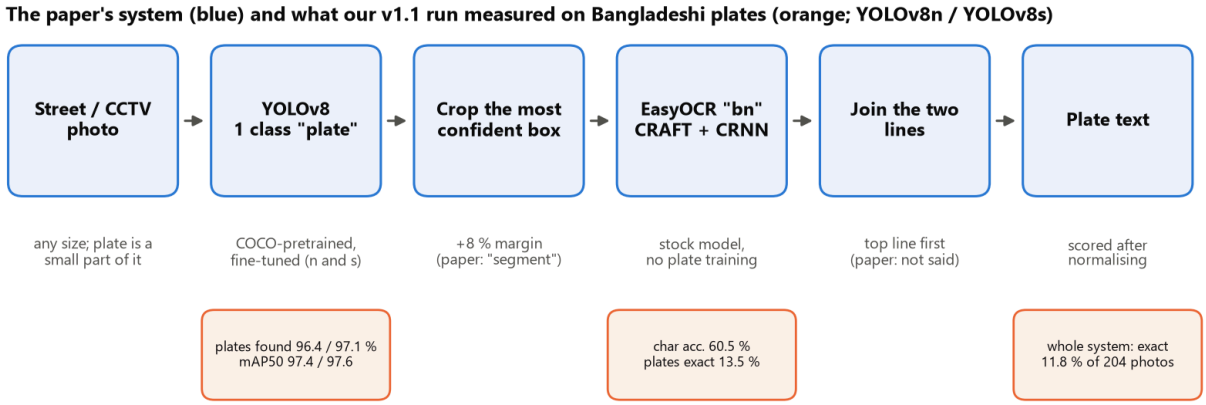


Figure 1. The paper's system (p.5-8) with what our v1.1 run measured at each step (orange: YOLOv8n / YOLOv8s). Sections 9-13 explain the numbers.

2. Key terms in plain words

Term	Meaning
YOLOv8 (n / s)	a one-stage object detector by Ultralytics (PyTorch, anchor-free). "n" (nano, 3.0 M parameters) and "s" (small, 11.1 M) are two sizes; the paper does not say which size it used
COCO-pretrained, fine-tuned	the model starts from weights learned on the 80-class COCO photos and is then trained on plate boxes only (one class)
IoU	intersection over union: how much a predicted box overlaps the label box (1 = perfect). IoU 0.5 = half overlap
Precision / recall	of the boxes the model draws, how many are real plates / of the real plates, how many get a box
mAP50 and mAP50-95	average precision with a box counted right at IoU 0.5 (mAP50), or averaged over IoU 0.50 to 0.95 (mAP50-95, rewards very tight boxes). The paper's "mAP 94.0" is mAP50 (its Fig. 6, p.11)
Plates found %	our plain number: share of human plate boxes with a model box at confidence >= 0.25 and IoU >= 0.5
EasyOCR	a free OCR library (JaidedAI): CRAFT finds text lines, a CRNN reads each line. The "bn" model was trained on general Bangla text, not on plates
Character accuracy / CER	CER = edits needed to turn the reading into the truth, divided by the truth length; char accuracy = 100 - CER. The paper never defines its "character recognition accuracy"
Plate exact	the whole plate text is right (after removing spaces and hyphens). One wrong symbol = wrong plate. The number a user cares about
Whole system (e2e)	photo in, plate text out: detector + crop + reader, scored on 204 photos with typed plate text
TensorRT FP16	NVIDIA's optimised runtime with 16-bit numbers; the fastest way to run a model on an NVIDIA GPU
McNemar test	compares two systems on the same plates using only the plates where exactly one is right; $p < 0.05$ = a real difference

3. The data

Item	Detail	Page
Photos	2,600 own photos, different light, angles, rain, dust, low light; about 90 % taken from the left or right side	p.6-7
Labels	boxes drawn by hand in Roboflow, one class "Bangladeshi License Plate"	p.7
Augmentation	blur, noise and weather effects in Roboflow, applied **before** the split; three "dataset propositions", the best model is kept	p.7, Fig. 3
Split	about 70 % train / 20 % val / 10 % test; test = 260 photos	p.7, p.9
Text labels for OCR	not described; Table 2 counts 100 clean + 80 low-resolution + 50 damaged + 30 weather plates = 260	p.12
Availability	not public, no link	-

Why "augmented before the split" matters. If a photo is copied with blur or noise and the copies are then split at random, the test set can contain near-copies of training photos. The model has then "seen" the test. Our own team results show how much this inflates scores (paper #23: 85.6 vs 77.8 % reading on the same plates). The paper itself lists "addressing the limitations of augmented datasets" as future work (p.17).

4. The method

Fig. 3 (p.6): augmentation BEFORE the split

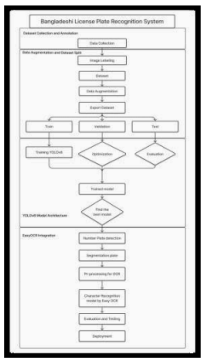


Fig. 6 (p.11): an Ultralytics results.png (mAP50 ≈ 0.95, mAP50-95 ≈ 0.59)

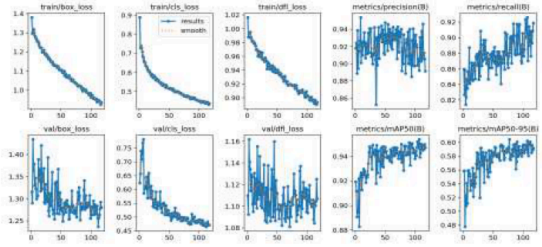


Fig. 8 (p.14): two lines, two results (letters 0.52, digits 0.96 confidence)



Figure 2. Three figures from the paper. Left: the workflow, with augmentation before the train/val/test split. Middle: the training plot is a standard Ultralytics results.png (about 118 epochs), although the text says TensorFlow. Right: EasyOCR returns the two plate lines as two separate results; the letters line has much lower confidence.

Step	What the paper says	Page
1. Detector	YOLOv8, pretrained on COCO, fine-tuned; "hyperparameters such as anchor boxes and learning rate" tuned (YOLOv8 has no anchors); model size, epochs, batch, image size not given	p.7
2. Crop	"YOLOv8 provides the bounding boxes"; the plate is "segmented to isolate individual characters" (not described)	p.8
3. Preprocessing	"preprocessing to enhance clarity" (not described)	p.8
4. Reading	EasyOCR, "specifically configured" for Bangla; no fine-tuning mentioned	p.8
Tools	Python, PyCharm, "TensorFlow" for YOLOv8, OpenCV, EasyOCR; training on Google Colab; Intel i5, 16 GB RAM	p.8

Our re-implementation fills the gaps with the most standard choice and writes each one down: YOLOv8n and YOLOv8s, 120 epochs (Fig. 6 shows about 118), 640 px, batch 16, SGD, seed 42; the most confident box is cropped with 8 % margin; EasyOCR `bn` reads it, and the lines are joined top line first. The paper's undefined "preprocessing" is tested as 4x upscale + grey + CLAHE (a common choice), next to the raw crop.

5. What the paper reports

Result	Value	Page
Detection accuracy / precision / recall / mAP	**94.8 / 93.5 / 92.3 / 94.0 %** on 260 test photos	p.9, Table 1
Training plot, last epoch (read off)	mAP50 about 0.95, mAP50-95 about 0.59, P about 0.92, R about 0.90, about 118 epochs	p.11, Fig. 6
Character recognition accuracy (EasyOCR)	**92.7 %**	p.11
... by case	clean 97.5 % (100 plates), low resolution 92.0 % (80), damaged 85.6 % (50), bad weather 89.3 % (30); weighted mean 92.6 %	p.12, Table 2
Adverse conditions (metric not named)	low light 91.5, rain 89.3, dust 87.2, partly hidden 85.0 %	p.13, Table 4
Speed and comparison	YOLOv8 78 FPS; YOLOv7 65, Faster R-CNN 42, SSD 85 FPS (other rows cited from other papers, other data; GPU not named)	p.12, Table 3
Abstract	"94.8 % detection and recognition accuracy" (two different things merged into one number)	p.1
Training time, model size	not reported	-

6. Weaknesses (with page numbers, so you can show them)

- **No metric is defined** (p.9, p.11): "detection accuracy" and "character recognition accuracy" could mean many things; the abstract merges them into one 94.8 % (p.1).
- **Augmentation before the split** (p.7, Fig. 3): copies of one photo can sit in train and test, which inflates every number.
- **No code, no data, no settings**: model size, epochs, image size, batch and learning rate are missing; the 2,600 photos are not public.
- **The text does not fit YOLOv8**: "TensorFlow" (p.8), but YOLOv8 is PyTorch and Fig. 6 is an Ultralytics plot; "anchor boxes" (p.7), but YOLOv8 is anchor-free; "Object Loss" (p.10, Fig. 5) has the values of the DFL loss.
- **Contradictions**: the discussion says Faster R-CNN has a higher mAP than YOLOv8 (p.13), Table 3 says the opposite (p.12); the conclusion says the work focuses "on detection rather than full character recognition" (p.17) while the abstract claims recognition accuracy; the YOLOv7 row cites the YOLOv1 paper (p.12).
- **Comparison table on other data**: Table 3 copies numbers from other papers (other photos, other hardware), so it is not a fair race.
- **EasyOCR is not explained**: no word on how the two plate lines are joined, though its own example (Fig. 8, p.14) shows two separate results.
- **Preprint**: not peer-reviewed.

> **Fair credit.** The recipe is reasonable and easy to repeat, it names real problems of Bangladeshi plates (two lines, Bangla script, CCTV), and it is the most direct published baseline for "YOLOv8 + general OCR". Treat the numbers as weak evidence and the recipe as a baseline; that is how we use it.

7. What we ran

The notebook `paper06_kaggle.ipynb` (built by `docs/_kit` like every team kit) runs the paper's method on the **498R BanglaLPR All-in-One v1.1** benchmark: all public Bangladeshi plate sets merged, near-duplicates kept in one group, grouped 70/15/15 split with seed 42, validation and test with human labels only.

Job	GPU	What it does	Time
`det_v8n`	0	YOLOv8n (COCO), 120 epochs, 640 px, batch 16, SGD, seed 42, on the 4,067 human-boxed training photos; tests, per-condition tests, plate counts	134.5 min
`det_v8s`	1	the same with YOLOv8s	147.7 min
`speed_det_v8n / v8s`	0	batch-1 speed on an idle T4: PyTorch FP32 / FP16, TensorRT FP16, CPU ONNX	5.2 / 3.0 min
`easyocr_crops`	0	stock EasyOCR 1.7.2 bn on the 651 human-typed OCR test crops: raw and with 4x/ grey/CLAHE; per condition, real fog copies, synthetic rain	9.3 min
`e2e_v8n / e2e_v8s`	1	whole system on the 204 e2e_test photos: detector → most confident box + 8 % → EasyOCR	0.7 min each

Data (v1.1)	Train	Val	Test
detection photos (human boxes used here)	4,067 human (+4,507 auto, not used)	896	1,153 (1,011 plates; 160 photos without a plate)
OCR crops with typed plate text	-	-	651 (+26 real fog copies, +651 synthetic rain copies)
e2e photos with plate text	-	-	204 (144 BLPD external + 60 NightLPBD)

Why a v1.1 re-run was needed. Dataset v1.1 changed only recognition and OCR crops: 19 BLPD test crops that were whole vehicle photos were cut to the plate. The detection split is identical to v1.0. So the detectors would repeat, but the EasyOCR test on the 651 crops had to be re-run, and it now is.

Differences from the paper (it gives no settings): our data instead of its 2,600 photos; both YOLOv8n and v8s; human boxes only (like its hand-drawn boxes); undefined metrics replaced by defined ones (mAP50, plates found %, char accuracy = 100 - CER, plate exact); "preprocessing" tested as one explicit recipe; Ultralytics 8.4.170 (PyTorch 2.11, CUDA 12.8, Python 3.13). The run started 2026-10-24 03:12 UTC on Tesla T4 x2.

8. Is the training OK?

Check	Result	OK?
Dataset version	`dataset_version: 1.1` in environment.json; downloaded from the team Drive; all counts as expected (log lines 34-72)	yes
Jobs	all 7 ended with code=0; 2.8 h of the 12 h limit (planned 3.5 h)	yes
Epochs	120 of 120 for both (time caps 3 h / 4 h not reached: 130.9 / 144.5 min of training)	yes
Learning	no NaN; validation mAP50 rises from 92.3 (epoch 1) to 97.7 (v8n); best epochs 90 (v8n) and 79 (v8s); the last epoch is within 1.0 (v8n) and 2.2 (v8s) mAP50-95 points of the best	yes
Over-fitting?	val box loss flat after about epoch 60, mAP50-95 flat; best.pt (the saved best epoch) is used everywhere, so a late dip does not matter	no problem
Repeatable (detectors)	local re-test of both best.pt on the same 1,153 test photos (RTX 5060 Ti): mAP50 97.39 / 97.62, mAP50-95 45.00 / 45.88, plates found 96.44 / 97.13 %: identical to Kaggle	yes
Repeatable (EasyOCR)	local re-run of EasyOCR 1.7.2: char accuracy 60.45 % (Kaggle 60.45), plate exact 13.67 % (Kaggle 13.52); 90 % of readings letter-for-letter the same (GPU and library versions differ)	yes
Whole-system crops	the local reader-swap test found plates in exactly the same share of photos as Kaggle (94.61 / 96.08 %)	yes

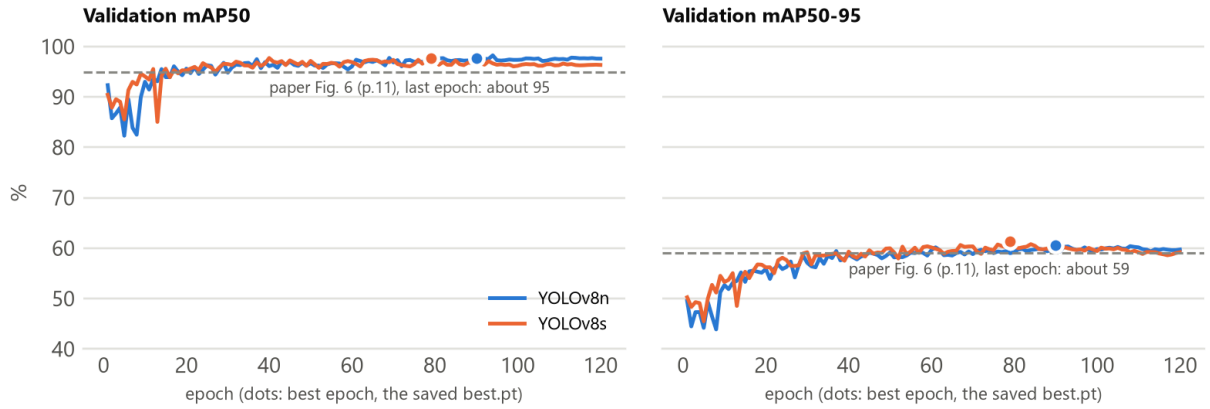


Figure 3. Validation mAP50 and mAP50-95 per epoch for both detectors (dots: the best epoch, saved as best.pt). Dashed: the last-epoch values read off the paper's own Ultralytics plot (Fig. 6, p.11). Our validation curves end at or above the paper's.

> **Validation 60.7 vs test 45.0 mAP50-95: is the model over-fitted to validation?** No. The validation set has 101 photos from the ridoykj source (11 %), the test set 274 (24 %). That source draws tighter boxes (exactly on the plate edge) than the others, and mAP50-95 punishes every pixel of disagreement. Section 10 shows it source by source: without ridoykj the test mAP50-95 is 64.3 (v8n), close to validation. mAP50 is the same on both (97.7 val, 97.39 test).

9. Results next to the paper

Metric	Paper (page)	YOLOv8n	YOLOv8s
Precision (%)	93.5 (p.9)	96.57	96.18
Recall (%)	92.3 (p.9)	94.07	94.71
mAP50 (%)	94.0 ("mAP", p.9)	**97.39**	**97.61**
mAP50-95 (%)	about 59 (Fig. 6, p.11)	44.97	45.88
mAP50-95 without the ridoykj source (%)	-	64.27	64.45
Plates found, conf 0.25, IoU 0.5 (%)	94.8 "detection accuracy" (p.9, undefined)	96.44	97.13
Validation mAP50 / mAP50-95 (%)	-	97.7 / 60.68	97.61 / 61.43
FPS, T4, batch 1: PyTorch FP32 / TensorRT FP16	78 (GPU not named, p.12)	113.8 / 182.9	109.6 / 162.5
Training time (min, T4) / parameters	not reported	130.9 / 3.01 M	144.5 / 11.14 M

EasyOCR and whole system	Paper (page)	Ours
EasyOCR char accuracy, 651 human crops (%)	92.7 (p.11, undefined)	**60.45**
EasyOCR plate exact (%)	not reported	**13.52**
EasyOCR with 4x / grey / CLAHE preprocessing: char acc / exact (%)	"preprocessing" (p.8)	41.80 / 4.92
EasyOCR clear / night / blurry crops: char accuracy (%)	clean 97.5 / low light 91.5 / - (p.12-13)	66.8 / 49.7 / 38.7
EasyOCR real fog / synthetic rain: char accuracy (%)	weather 89.3 / rain 89.3 (p.12-13)	59.5 / 44.0
EasyOCR low-resolution plates	92.0 (p.12)	no crop in our OCR test is under 24 px high (rule of the condition tag), so not measured
EasyOCR ms per plate (T4)	not reported	90.9
Whole system, 204 photos: plate found / char acc / exact (%), v8n	-	94.61 / 58.34 / **11.76**
Whole system, v8s	-	96.08 / 57.64 / **10.29**
Whole system ms per photo (detector + EasyOCR, PyTorch)	-	68.7 (v8n), 69.5 (v8s)

The predictions we wrote down before the run (notes, 11 and 21 Oct): were they right?

What	Expected	Result	Why
mAP50, precision, recall	lower than the paper	**not confirmed**: higher (mAP50 97.4 / 97.6 vs 94.0)	more training photos (4,067 vs about 1,820), mostly large plates in our photos (739 of 1,011 test plates are "large"), clean human boxes
mAP50-95	lower than about 59	confirmed on paper (45.0), **but** 64.3 without ridoykj	one source's box style (section 10)
FPS	v8n faster than 78	confirmed: 114 (FP32) / 183 (TensorRT FP16)	T4, batch 1, pre- and post-processing included
EasyOCR character accuracy	much lower than 92.7	confirmed: 60.5 %; plate exact 13.5 %	general Bangla model, not trained on plate fonts or two-line layout
By condition	clear better than night / blur / fog	confirmed: clear 66.8 > fog 59.5 > night 49.7 > rain 44.0 > blurry 38.7	as the paper's Table 2, only much lower
YOLOv8n vs v8s	s slightly better, n about 2x faster	partly: s +0.2 mAP50, 17 vs 10 plates (p = 0.25, a tie); n only 1.04x faster in FP32, 1.13x with TensorRT	at batch 1 the fixed pre- and post-processing time dominates

Our photos and the paper's are different, so the rows show direction and size, not a race. The comparison that *is* fair is ours against other models on the **same** test plates (sections 10, 11 and 13).

10. The detector in detail

Why mAP50-95 is 45 and not 59: one source boxes tighter

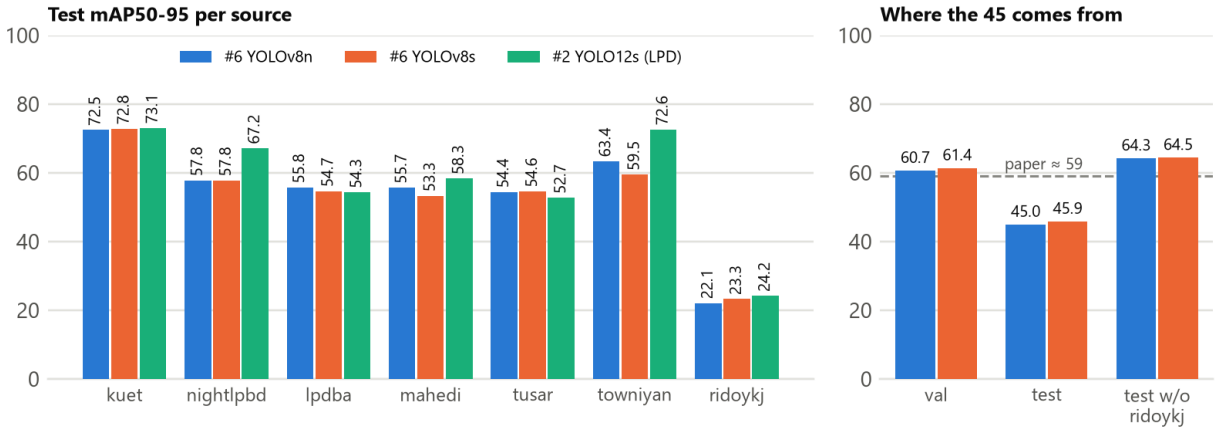


Figure 4. Left: test mAP50-95 for each source, for both #6 detectors and paper #2's YOLO12s plate detector (all three trained on our v1.1 split). Every model scores 22-24 on ridoykj and 52-73 elsewhere. Right: validation, test, and test without ridoykj.

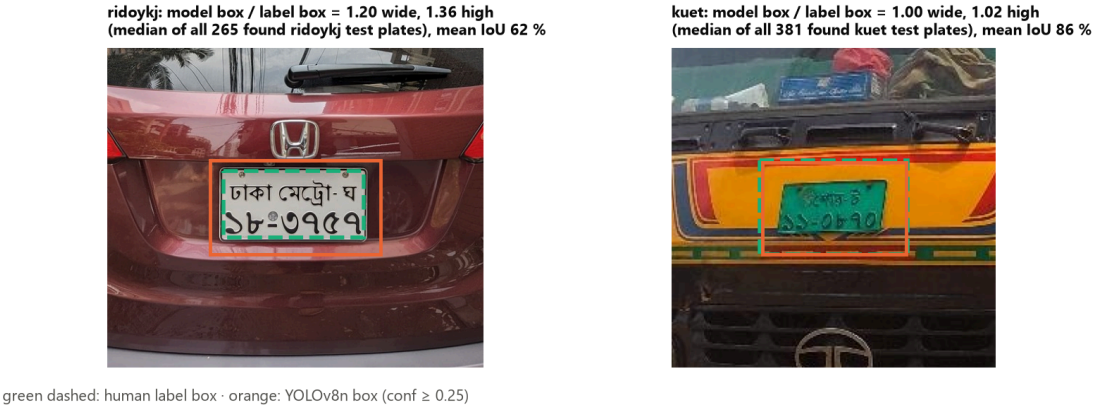


Figure 5. A ridoykj and a KUET test photo with the human label (green, dashed) and the YOLOv8n box (orange). Ridoykj labels follow the plate edge exactly; the model, trained mostly on sources that box the plate with a margin, draws a larger box. Both boxes find the plate (IoU above 0.5), but mAP50-95 also counts the IoU 0.75-0.95 levels, where they disagree.

All three detectors find almost every ridoykj plate (mAP50 96.4-99.5), but their boxes are 1.20x as wide and 1.36x as high as the ridoykj labels (median), against 1.00 / 1.02 on KUET. Mean IoU is 62.0 % on ridoykj vs 85.9 % on KUET. Paper #2's report found the same for its own detector (24.2 on ridoykj), so this is a property of the labels, not of YOLOv8. **Lesson for our evaluation:** report mAP50 and plates found as the main detector numbers, and mAP50-95 per source or without ridoykj.

By condition

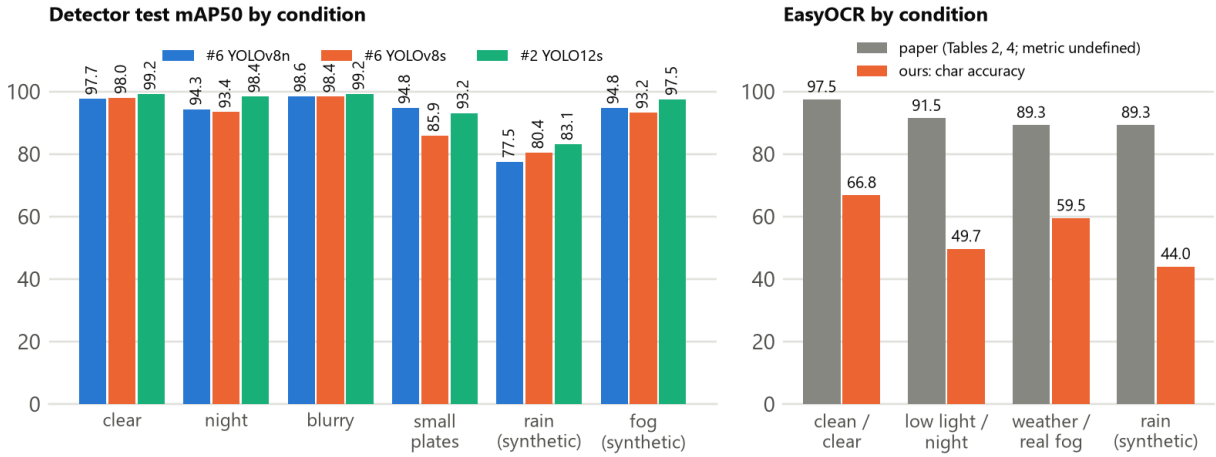


Figure 6. Left: detector test mAP50 on the fixed team condition subsets (clear 868 photos, night 71, blurry 53, small plates 22; rain and fog are synthetic copies of the test). Right: EasyOCR character accuracy on our closest condition subsets next to the paper's Tables 2 and 4 (its metric is undefined and on its own photos).

- **Clear, blurry and synthetic fog are easy** for both detectors (mAP50 97.7, 98.6, 94.8 for v8n).
- **Synthetic rain is the hardest**: 77.5 / 80.4 mAP50, and only 64 % of plates found by v8n. The paper's rain claim (89.3 %) is not supported on our rain copies.
- **Night**: 94.3 / 93.4 mAP50; on NightLPBD v8n finds 80 % of plates. Paper #2's detector, trained with the auto-labelled photos too, finds 85 % (5 vs 2 plates, p = 0.45).
- **Small plates** (22 photos): v8n 94.8, v8s 85.9. Few photos, so the v8n-v8s gap there is noise. Only 5 test plates are COCO-small (under 32x32 px).
- **False alarms**: a box on 6.25 % of the 160 plate-free photos (both sizes).

Against other detectors on the same test photos

Detector (all on the v1.1 detection split)	Training boxes	mAP50	mAP50-95	Plates found	Same plates vs #6 YOLOv8n
#6 YOLOv8n (this run)	human, 120 epochs	97.39	44.97	96.44 %	-
#6 YOLOv8s (this run)	human, 120 epochs	97.61	45.88	97.13 %	17 vs 10 plates, p = 0.25 (tie)
#3 YOLOv8n (Md. Yousuf)	human + auto, 60 epochs	98.73	45.23	98.12 %	not paired (no per-plate file); same photos
#2 YOLO12s plate detector (our #2 run)	human + auto, 60 epochs	98.87	46.18	98.32 %	24 vs 5 plates, p = 0.0005

> **What this says.** (1) YOLOv8n and YOLOv8s are tied on our data: take n for speed. (2) The same YOLOv8n trained by a teammate with the **auto-labelled** photos as well (DIU, LPD4K, BLPD: 4,507 more photos) finds more plates (98.1 vs 96.4 %) in half the epochs, and paper #2's YOLO12s with the same boxes beats our v8n on the same plates (24 vs 5, p = 0.0005). The paper's "human boxes only" choice costs about 1.5-2 points of plates found; for the final detector, train on human + auto boxes.

11. EasyOCR: why the reading fails

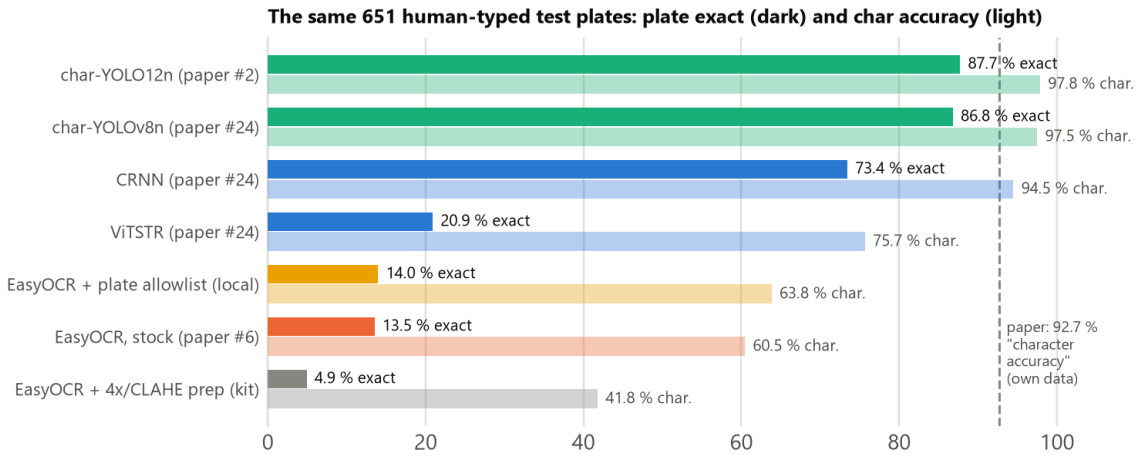


Figure 7. The same 651 human-typed test crops read by EasyOCR (this paper) and by the plate readers our team trained on the same v1.1 data (papers #2 and #24). Dark: plate exact; light: character accuracy. Dashed: the paper's 92.7 % on its own data.

On the same 651 crops, paper #2's character detector reads 87.7 % of plates exactly, EasyOCR 13.5 %: the detector is right on **487** plates where EasyOCR is wrong, EasyOCR on 4 where the detector is wrong ($p < 0.0001$). Even our weakest trained reader, ViTSTR, beats it (20.9 %, $p = 0.0007$). EasyOCR reaches the paper's level on no subset.



Figure 8. Five test crops, one per typical case (readings in the table below).

Case	Truth	EasyOCR (confidence)	Paper #2 char-YOLO12n
(a) right	ঢাকা মেট্রো-ব ১৫-৬৬২৬	ঢাকা মেট্রো: ব ১৫-৬৬২৬ (0.69)	ঢাকা মেট্রো-ব ১৫-৬৬২৬ ✓
(b) Latin digits	ঢাকা মেট্রো-গ ২৭-৭৪৪২	ঢাকা মেট্রোগ ২৭-৭৪৪২ (0.76)	ঢাকা মেট্রো-গ ২৭-৭৪৪২ ✓
(c) metro word broken	রাজ মেট্রো-হ ১২-৩৬৫৩	রাজ মেত্রো- হ ১২-৩৬৫৩ (0.91)	রাজ মেট্রো-হ ১২-৩৬৫৩ ✓
(d) night	যশোর-ট ১১-৪২৬৭	যশরটি১৯৪২৬৭ (0.79)	যশোর ৩১১৯৪২৫৬৭ ✗
(e) junk	ঢাকা মেট্রো-গ ২৯-৭৩৭০	5141 :ম'!| ২৯-৭৩৭0 (0.34)	ঢাকা মেট্রো-গ ২৯-৭৩৭০ ✓

Where the errors are

Check (651 crops)	Result	What it means
Letters line right (district, মেট্রো, series letter)	19.2 %	the Bangla words are the main problem
Digits line right (6 digits)	32.7 %	digits are easier, but still wrong on 2 of 3 plates
Readings with English digits (0-9)	39.2 %	৪ is read as "8", ০ as "0": the bn model mixes scripts; scoring maps them by value, so "8" counts as ৮, a wrong digit
Readings with brackets, bars or other punctuation	40.9 %	plate frame, bolts and dirt read as characters
Characters missed most (share of their occurrences)	গ 56 %, চ 54 %, ো 54 %, ট 50 %, ব 50 %, হ 47 %	series letters (গ ব হ) and the ো and ট of মেট্রো; most common swap: ৪ read as "8" (55 times)
Empty readings	1.8 %	CRAFT found no text
Valid BRTA format	8.8 % of readings; of those, 84.2 % are exactly right	a format check is a strong "trust this reading" signal
Confidence as a warning	AUC 0.89; mean 0.78 on right vs 0.45 on wrong readings; at confidence >= 0.9: 67 % right, but only 4.6 % of plates kept	confidence ranks errors well, but almost nothing is both confident and right

The paper's "preprocessing" makes it worse

With 4x upscale + grey + CLAHE, exact reading drops from 13.5 to 4.9 % (69 plates lost, 13 gained, $p < 0.0001$) and the time per plate rises from 90.9 to 580.9 ms. Our crops are already large (median 253 px high, 10 % below 150 px), so a 4x upscale makes huge images, CRAFT then splits the text into more pieces, and CLAHE strengthens dirt and frame edges. Preprocessing has to be tested, not assumed (proposal G4).

By source and condition

Subset	Crops	Char accuracy	Plate exact
source anpr	184	73.0 %	23.4 %
source lpdba	202	66.7 %	18.3 %
source blpd	30	47.9 %	3.3 %
source kuet	183	46.9 %	3.3 %
source nightlpbd	52	47.8 %	1.9 %
condition clear	480	66.8 %	17.7 %
condition night	65	49.7 %	1.5 %
condition blurry	112	38.7 %	1.8 %
real fog copies	26	59.5 %	15.4 %
synthetic rain copies	651	44.0 %	5.7 %

EasyOCR is best on the ANPR and LPDB-A crops and worst on KUET, BLPD and night photos. At night and on blurry crops almost no plate is read exactly (1.5 % and 1.8 %). The paper's 91.5 % for "low light" and 89.3 % for rain cannot be reproduced on real plates.

12. Can EasyOCR be saved cheaply? (local check)

Before dropping EasyOCR, we tried the two cheapest fixes on a local GPU (RTX 5060 Ti, EasyOCR 1.7.2, same 651 crops, same line joining as the kit; `report_src/easyocr_check.py`): an **allowlist** (EasyOCR may only output the 59 characters that occur in our training plate texts, so no "8", "|" or "[") and a mild 2x upscale.

Variant	Plate exact %	Char acc. %	Valid format %	ms / plate (local)	vs local raw (plates gained vs lost)
raw crop (= the Kaggle job)	13.67	60.45	8.91	56.1	-
4x + grey + CLAHE (kit)	5.07	41.76	3.07	325.5	13 vs 69, p < 0.0001
2x upscale, colour	13.06	57.47	7.83	101.0	24 vs 28, p = 0.68
allowlist	13.98	63.83	5.99	51.5	10 vs 8, p = 0.81
allowlist + 2x upscale	13.06	61.66	4.45	101.1	28 vs 32, p = 0.70

> **Result: no cheap fix works.** The allowlist removes the English digits and raises character accuracy by 3.4 points (60.5 → 63.8 %), but exact reading stays where it was: 13.98 vs 13.67 % (10 plates gained, 8 lost, p = 0.81). The 2x upscale is slightly worse, with or without the allowlist, and doubles the time. So the problem is not the output alphabet or the image size but the model: the bn recogniser has never seen two-line plate layouts, plate fonts or মেট্রো, and when it is forced to pick a Bangla character instead of "8" it often picks the wrong one. A character detector trained on our plates reads 87.7 % of the same crops. Fine-tuning EasyOCR on our 5,607 training plate texts would be the fair test of "EasyOCR adapted to plates" (section 21).

13. The whole system: detector or reader?

The kit's whole-system test reads the 204 e2e photos with YOLOv8 + EasyOCR. To see which part loses the plates, we kept **exactly the same detector crops** (most confident box, 8 % margin) and replaced only EasyOCR with paper #2's character reader (char-YOLO12n, trained on the same v1.1 data), locally (`report_src/local_check.py`). We also put the #6 detectors into paper #2's cascade code in place of its own plate detector.



Figure 9. What happens to the 204 photos. Grey: the detector found no plate; orange: plate found but read wrong; green: read exactly. The detector loses 8-11 photos in every system; EasyOCR loses most of the rest.

System (204 photos)	Plate found	Exact	Char acc.	BLPD ext. / night exact	ms per photo
#6 YOLOv8n + EasyOCR (Kaggle)	94.61 %	**11.76 %**	58.3 %	15.3 / 3.3 %	68.7 (T4)
#6 YOLOv8s + EasyOCR (Kaggle)	96.08 %	**10.29 %**	57.6 %	13.2 / 3.3 %	69.5 (T4)
#6 YOLOv8n + char-YOLO12n, same crops (local)	94.61 %	**84.80 %**	94.0 %	93.8 / 63.3 %	31.9 (local)
#6 YOLOv8s + char-YOLO12n, same crops (local)	96.08 %	**88.24 %**	95.2 %	93.8 / 75.0 %	28.6 (local)
#2 cascade with #6 YOLOv8s (local)	96.08 %	88.24 %	95.2 %	93.8 / 75.0 %	24.3 (local)
#2 cascade with its own YOLO12s (local = Kaggle)	95.59 %	86.76 %	94.9 %	93.8 / 70.0 %	30.2 (local)

- **The reader is the bottleneck.** Same photos, same crops: the char-YOLO reader is right on 151 photos where EasyOCR is wrong, EasyOCR on 2 ($p < 0.0001$).
- **The #6 detector is good enough.** In paper #2's cascade, YOLOv8s gives 88.24 % vs 86.76 % with #2's own YOLO12s (7 vs 4 photos, $p = 0.55$: a tie). YOLOv8n: 85.29 % ($p = 0.61$).
- **Best combination so far** on these 204 photos: #6 YOLOv8s + #2 char-YOLO12n, 88.24 % exact (night 75.0 %). It is not significantly better than #2's own cascade, so call it "as good as".
- **Lost before reading:** 11 photos (v8n) and 8 (v8s) have no plate box at all, every one a NightLPBD night photo (#2's detector: 9); this is the detector's share of the loss.
- The local check reproduces #2's Kaggle cascade photo by photo (the same 177 photos right), so the swap test is trustworthy.

14. Speed

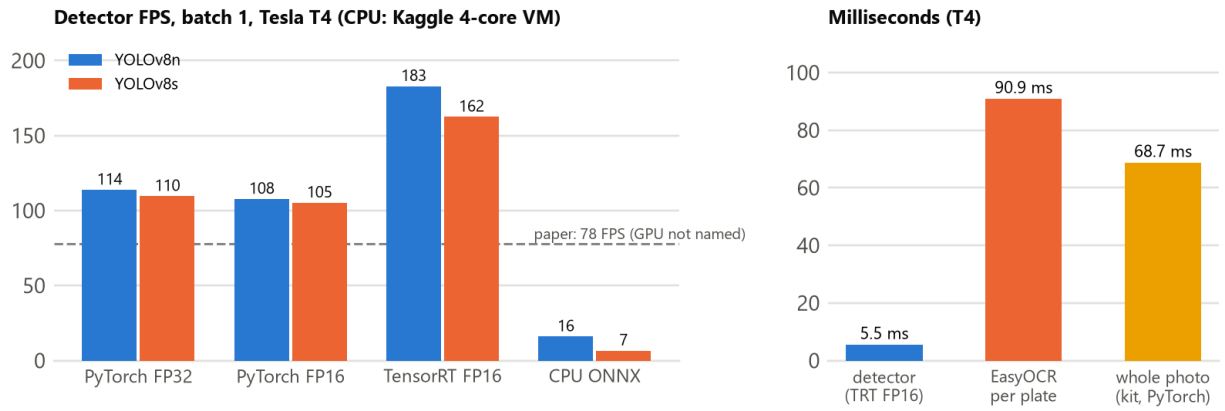


Figure 10. Left: detector frames per second at batch 1 on one Tesla T4 (CPU: the Kaggle VM's 4 cores, ONNX Runtime), pre- and post-processing included. Dashed: the paper's 78 FPS (GPU not named). Right: where the time of one photo goes.

Speed, batch 1	YOLOv8n	YOLOv8s	Paper
T4 PyTorch FP32: FPS (ms)	113.8 (8.78 ms)	109.6 (9.12 ms)	78 FPS (GPU and precision not named)
T4 PyTorch FP16	107.7	105.2	-
T4 TensorRT FP16	**182.9** (5.47 ms)	**162.5** (6.15 ms)	-
CPU ONNX (Kaggle, 4 cores)	16.0	6.6	-
Parameters / GFLOPs	3.01 M / 8.2	11.14 M / 28.6	not given
EasyOCR per plate (T4, raw crop)	90.9		-
Whole photo, detector + EasyOCR (T4, PyTorch)	68.7 ms = 14.6 photos/s	69.5 ms	-
Char-YOLO readers per plate (T4; papers #2 / #24)	23.6 / 9.4 ms		-

Reading the table. PyTorch FP16 is not faster than FP32 at batch 1 for such small models (the GPU waits for Python); TensorRT is. v8n is only 1.04x faster than v8s in FP32 because pre- and post-processing take the same time for both. The detector is not the speed problem of this system: EasyOCR takes 17x the TensorRT detector time per plate. On the Kaggle CPU, v8n reaches 16 FPS, enough for our 10 FPS target (RQ3) if the reader is light too.

15. Findings, and the numbers to quote

#	Finding	Number to quote	Section
1	The v1.1 run is healthy and repeatable	7/7 jobs code=0; 120/120 epochs; local re-test identical (mAP50 97.39)	8
2	The YOLOv8 detector is as good as the paper claims, and meets our 95 % target	mAP50 97.4 / 97.6 % (paper 94.0); plates found 96.4 / 97.1 %	9, 10
3	Low mAP50-95 is a label-style effect of one source	ridoykj 22.1; without it 64.3 (paper about 59)	10
4	Auto-labelled photos help the detector	human+auto v8n (#3) 98.1 % vs human-only 96.4 % found; #2 YOLO12s beats v8n 24 vs 5 plates (p = 0.0005)	10
5	Stock EasyOCR cannot read Bangladeshi plates	char acc. 60.5 % (paper 92.7); exact 13.5 %; letters line 19.2 %	11
6	Preprocessing hurts EasyOCR; cheap fixes do not help	4x/CLAHE 4.9 %; allowlist 14.0 % (p = 0.81) vs char-YOLO 87.7 %	11, 12
7	With a plate-trained reader the same system works	same 204 photos and crops: 11.8 → 84.8 % (v8n), 10.3 → 88.2 % (v8s)	13
8	Speed is not the issue; EasyOCR is the slow part	v8n 183 FPS TensorRT, 114 FPS FP32 (paper 78); EasyOCR 90.9 ms/plate	14

What we keep and what we drop

From paper #6	Decision	Why
YOLOv8 one-class plate detector	**keep** as a detector candidate (O2): v8n for speed, trained on human + auto boxes	findings 2, 4, 8
Stock EasyOCR bn	**drop** from the system; keep as the baseline row of our results table	findings 5, 7
"Preprocessing" (4x + grey + CLAHE)	**drop**; any enhancement goes into the G4 ablation with measured gains	finding 6
mAP50-95 as the headline detector number	**replace** with mAP50 + plates found; mAP50-95 per source	finding 3
Undefined "character accuracy"	**replace** with char accuracy = 100 - CER and plate exact	section 11
Allowlist / format check on OCR output	**keep the idea** for our decoder (O3: BRTA-format-aware correction)	section 11-12: valid format → 84 % right

16. How to use this paper in our research

Our research part (proposal)	How paper #6 is used	Evidence we now have (v1.1)
Baselines section ("EasyOCR after our detector")	it IS the promised baseline: YOLOv8 + stock EasyOCR, run on our benchmark	det mAP50 97.4; EasyOCR 60.5 % char acc., 13.5 % exact; e2e 11.8 %
RQ2 / O3: character-level recognizer, >= 90 % char accuracy	the general-OCR side of the comparison; shows why a plate-trained, character-level reader is needed	same 651 crops: char-YOLO 97.8 % char acc. / 87.7 % exact vs EasyOCR 60.5 / 13.5 % (487 vs 4, p < 0.0001)
O2: detector choice (YOLOv8 / YOLO11 / YOLOv12)	YOLOv8n and v8s rows with speed; n = s on accuracy	v8n 183 FPS TensorRT; v8s vs v8n 17 vs 10 plates (tie)
G3 / O1: benchmark and labels	auto-labelled photos are worth using; label style differs per source	human+auto 98.1 vs human 96.4 % found; ridoykj mAP50-95 22.1
RQ4 / G4: conditions, CLAHE and upscaling study	first measured data points	rain 77.5 mAP50; EasyOCR night 1.5 % exact; 4x+CLAHE −8.6 points
RQ3 / G1: >= 10 FPS	detector cost vs reader cost	detector 5.47 ms (T4 TRT), EasyOCR 90.9 ms; CPU v8n 16 FPS
O5: evaluation protocol	defined metrics and per-source reporting	undefined "accuracy" in the paper; mAP50-95 depends on label style
Demo system	detector part of the demo	YOLOv8s + #2 char-YOLO12n: 88.2 % exact on 204 photos (= #2 cascade 86.8 %, tie)

> **Fix one sentence in our proposal.** The related-work paragraph says Ahamed and Ismail "report 94.8 % combined detection and recognition accuracy". Keep the citation, but add what we measured: on our benchmark the same pipeline reads 13.5 % of plates exactly (60.5 % character accuracy).

17. What benefit do you get from it?

- **A finished, honest baseline.** The proposal promised "EasyOCR after our detector" as a baseline; it now exists with defined metrics, per-condition results and released weights.
- **A strong argument for our design.** Same photos, same crops, only the reader swapped: 12 % → 85-88 %. That one paired number justifies the character-level reader (RQ2) better than any citation.
- **A ready plate detector.** YOLOv8n: 6.2 MB, 97.4 % mAP50, 183 FPS with TensorRT on a T4, 16 FPS on a 4-core CPU; usable in the demo today.
- **An explained anomaly.** The low mAP50-95 is traced to one source's box style, which protects every teammate's detector numbers from a wrong conclusion.
- **Evidence that preprocessing must be measured**: the obvious "upscale + CLAHE" cuts EasyOCR's exact rate to about a third (13.5 → 4.9 %).
- **Practice in reading weak papers**: undefined metrics, augmentation before the split and a TensorFlow claim for a PyTorch model are exactly what examiners ask about.
- **Pre-registered predictions**: we wrote the expected gaps down before the run; 4 of 6 held, 1 held partly (v8n vs v8s speed), and the one that failed (the detector is better than the paper, not worse) has a reason (section 9).

A paragraph you can adapt for the related-work chapter

```
Ahamed and Ismail [ref] fine-tune a COCO-pretrained YOLOv8 as a plate detector and read
the crop with stock EasyOCR, reporting 94.0% mAP and 92.7% character accuracy on 260 own
test photos; the metrics are not defined and the photos were augmented before splitting.
Re-implemented on our duplicate-free benchmark, the detector reaches 97.4% mAP50
(YOLOv8n), but stock EasyOCR reads only 60.5% of the characters and 13.5% of 651 plates
exactly, and neither an output allowlist nor upscaling changes that. With the same
detector crops, a character-level YOLO reader trained on our data reads 84.8-88.2% of
204 whole photos exactly, against 10-12% with EasyOCR.
```

18. What to show the faculty (8 slides)

#	Slide title	What is on it	Source in this report
1	The paper and its claim	YOLOv8 + stock EasyOCR; 94.0 % mAP, 78 FPS, 92.7 % char accuracy on 260 own photos; no code, no data	sections 1, 5
2	What we checked first	undefined metrics, augmentation before the split, TensorFlow/anchor contradictions (with page numbers)	section 6, Figure 2
3	Our setup	v1.1 benchmark, human boxes, YOLOv8n/s, 120 epochs, T4 x2; training healthy and repeatable	sections 7-8, Figure 3
4	Detector: as good as claimed	mAP50 97.4 / 97.6; 114 / 183 FPS; mAP50-95 explained by ridoykj	section 9-10, Figures 4-5
5	EasyOCR: the weak link	60.5 % char accuracy, 13.5 % exact; examples; English digits; preprocessing hurts	section 11, Figures 7-8
6	The key experiment	same photos and crops, reader swapped: 11.8 → 84.8 % (v8n), 88.2 % (v8s)	section 13, Figure 9
7	Conditions and speed	rain and night are the hard cases; EasyOCR is 17x the detector time	Figures 6, 10
8	What we take for our system	YOLOv8n detector (human + auto boxes); character-level reader; EasyOCR stays a baseline; defined metrics	section 15 table

> **Keep it honest:** call 97.4 % "mAP50 on our test split" and never claim we "beat" the paper (different photos). Say "same photos" only for comparisons on our own test (sections 10, 11, 13). The reader-swap and EasyOCR-fix tests ran on our own PC, not on Kaggle; say so if asked. The #3 comparison (auto labels) is not paired.

19. What to explain: a 3-minute script

"This 2025 preprint builds the simplest plate reader for Bangladesh: a YOLOv8 detector fine-tuned to find plates, and the free EasyOCR Bangla model to read them. It reports 94 percent mAP, 78 frames per second and 92.7 percent character accuracy. But it is a preprint without code or data, its metrics are not defined, and it augmented the photos before splitting them, so test photos can be copies of training photos.

We rebuilt it on our own benchmark, where duplicates are kept in one split. Both YOLOv8n and YOLOv8s trained cleanly for 120 epochs, and a local re-test gives the same numbers. The detector is as good as claimed: 97.4 percent mAP50, 96 percent of plates found, 183 frames per second with TensorRT. Its mAP50-95 looks low, 45, but that comes from one source whose labels sit exactly on the plate edge while the others leave a margin; without it, the score is 64.

The reading is the problem. Stock EasyOCR reaches 60 percent character accuracy and reads only 13.5 percent of 651 real plates exactly. It mixes in English digits, breaks the word Metro, and the paper's kind of preprocessing makes it worse. Forcing it to output only plate characters does not help either: still 14 percent.

Our key test keeps the same detector and the same crops on 204 whole photos and only swaps EasyOCR for the character-level reader we trained in paper 2: exact reading jumps from 12 to 85 percent with YOLOv8n and 88 percent with YOLOv8s. So in our system we keep YOLOv8 as the detector, train it on all our boxes, use a character-level reader, and keep EasyOCR only as the baseline we must beat."

20. Questions the faculty may ask

Q1. Your mAP50 is higher than the paper's. Did you beat it?
No claim of that: different photos. Ours has more training photos (4,067 human-labelled), mostly large plates, and a duplicate-free test; theirs is unknown and possibly leaky. It shows that YOLOv8 is a strong plate detector, which is all we need from it.

Q2. Your mAP50-95 is 45, theirs about 59. Is your box regression bad?
No. The ridoykj source (274 test photos) boxes the plate exactly at its edge, while the other sources leave a margin, so the model's boxes are about 1.2x wider and 1.4x higher than ridoykj's labels. Every detector we trained, including paper #2's YOLO12s, scores 22-24 there and 52-73 on the other sources. Without ridoykj our test mAP50-95 is 64.3, and validation (fewer ridoykj photos) is 60.7.

Q3. EasyOCR gets 92.7 % in the paper and 60 % for you. Is your EasyOCR set up wrong?
We use the same library and language as the paper, without fine-tuning, as it says. A local re-run on another GPU gives the same 60.5 %. The paper never defines its metric and tests on its own unpublished plates; its own example (Fig. 8, p.14) already shows low confidence (0.52) on the letters line, which is where we see most errors.

Q4. How do you join EasyOCR's two lines? Maybe that causes the errors.
Top line first, left to right, then scoring ignores spaces and hyphens. Even each line alone is mostly wrong: the letters line is right on 19.2 % of plates and the 6 digits on 32.7 %, so joining is not the cause.

Q5. Why not fine-tune EasyOCR?
That is a different method from the paper (which uses it stock). It is on our next-step list as a fair "EasyOCR on plates" test; our trained readers already reach 87.7 % exact on the same crops, so it has a high bar to clear.

Q6. Is the reader-swap test fair?
Yes: the same 204 photos, the same detector, the same crop for every photo; only the reader differs. Plates found is identical to the Kaggle run (94.61 %), and our local copy of paper #2's cascade reproduces its Kaggle result photo by photo.

Q7. YOLOv8n or YOLOv8s?
n. They tie on accuracy (17 vs 10 plates, p = 0.25), n has 3.7x fewer parameters and is faster on CPU (16.0 vs 6.6 FPS).

Q8. Does the detector work at night and in rain?
Night is weaker (v8n finds 80 % of NightLPBD plates; #2's detector with auto labels 85 %). Synthetic rain is the hardest case (77.5 mAP50). We have no real rain photos; that is a data gap for RQ4.

Q9. 78 FPS vs your 114-183: is that fair?
The paper names no GPU and no precision. Ours is batch 1 on a T4 with pre- and post-processing included, given per precision. Either way the detector is fast; the reader decides the system speed.

Q10. What is the one thing you take from this paper?
That a YOLOv8 plate detector is enough, and a general OCR is not: the paired swap test (12 % → 85-88 %) is the evidence for our character-level reader.

21. Next steps (in order)

1. **Upload** (section 22). The team table is filled: `python add_results.py paper06_results.zip --member "Rokib Hasan Oli"` in `docs/results` (done on 24 Oct; check the notes' results block).
2. **Detector for the system:** retrain YOLOv8n on human + auto boxes (as #3/#2 did) with our kit and compare on the same plates; add YOLO11n (O2). Report mAP50, plates found, per-source mAP50-95.
3. **Fine-tune EasyOCR's recogniser** on our 5,607 training plate texts (two lines as one image, or line by line) as the fair "general OCR adapted to plates" baseline; compare with char-YOLO on the same 651 crops.
4. **Decoder with BRTA format and district dictionary** (O3): the format check already marks good readings (84 % right when valid); use it to choose between readers or frames.
5. **Night and rain:** collect real rain photos; test night-specific augmentation for the detector (NightLPBD found rate 80 %).
6. **Speed on a laptop CPU** (RQ3): YOLOv8n ONNX/OpenVINO + char-YOLO reader end to end; target 10 FPS.
7. **Proposal text:** update the EasyOCR sentence in related work (section 16).

22. GitHub and Drive

`After Train/results.zip` (368 MiB) is the whole Kaggle output; do not upload it. The upload folder is ready at `After Train/github_upload/weights/paper06/MD.Rokib.Hasan.Oli/` (same layout as papers #2, #8, #23 and #24):

File	What
`README.md`	results, model cards, quick start (tested), checksums
`best_det_v8n.pt`	6.2 MB, YOLOv8n plate detector (best epoch 90)
`best_det_v8s.pt`	22.5 MB, YOLOv8s plate detector (best epoch 79); under GitHub's 25 MB web-upload limit
`paper06_results.zip`	2.4 MB, all results, logs, per-plate EasyOCR and whole-system predictions, code; no weights
`results_vs_paper.md`	the kit's ours-vs-paper table
`Paper06_Study_Report.pdf`	this report

Drive `498R / weights / paper06 /`: `After Train/for_drive/paper06_outputs.zip` (124 MiB: both models with best.pt, last.pt and ONNX exports). Web upload on GitHub: open `weights` → Add file → Upload files → drag the `paper06` folder → commit message "Add paper #6 results on dataset v1.1 (Rokib)".

Appendix: where every number comes from

Number	Source
paper numbers	the paper PDF (Papers/6_1 .../06_2025_YOLOv8-Based LPR for Bangladeshi Vehicles.pdf), pages given in the text; figures in Figure 2 extracted from it
v1.1 training, detector, EasyOCR, whole-system and speed results	`After Train/results.zip` → `runs/*/results.json`, `results.csv`, `ocr_preds_test*.csv`, `e2e_preds.csv`; `paper-6.log`
line analysis, error types, confidence, paired tests with papers #2, #8, #24 (their v1.1 runs), #3 numbers	`report_src/analysis.py` → `analysis.json`
per-source detector scores, same-plate detector tests, reader swap on 204 photos	`report_src/local_check.py` → `local_check.json`, `det_per_plate.csv`, `e2e_cross_preds.csv` (RTX 5060 Ti, the run's best.pt, paper #2's uploaded weights, dataset v1.1)
EasyOCR local re-run and cheap fixes	`report_src/easyocr_check.py` → `easyocr_check.json`, `easyocr_local_preds.csv` (EasyOCR 1.7.2, bn)
charts	`report_src/charts.py`

```
@misc{ahamed2025yolov8,
  title        = {{YOLOv8}-Based License Plate Recognition for
                  {Bangladeshi} Vehicles},
  author       = {Ahamed, Istiak and Ismail, Gazi Mohammad},
  year         = {2025},
  month        = jan,
  howpublished = {Authorea preprint},
  doi          = {10.22541/au.173748276.69471512/v1},
  note         = {Not peer-reviewed}
}
```